

Multimodal AI Agent for Automated Educational Content and Interactive Companion Learning

Core Engine - Parallel GPU Inference Pipeline

Version 0.1

Lu Li (lu.li02@sjsu.edu)

Version History

Version	Changes
0.1	Initial draft for parallel GPU inference pipeline (core engine)

Introduction

This document describes the design of the Parallel GPU Inference Pipeline, which is my primary contribution to the EduTok Engine project. The goal of this component is to enable scalable and low-latency video generation by leveraging multiple GPUs. Based on our project requirements, the target latency is less than 15 seconds per generated video clip.

At the current stage of the project (as of April 2026), the system is still under active development and some components described here are design-level and not fully implemented yet. However, I have completed the local GPU environment setup (RTX 4090 workstation), conducted initial experiments with AnimateDiff, and identified performance bottlenecks in sequential inference. This design is based on those observations and ongoing HPC setup efforts.

References

- DeepSpeed ZeRO (GPU scaling)
- AnimateDiff / Stable Video Diffusion
- Sora

Requirements

1. Functional Requirements

- Process multiple scripts segments concurrently
- Assign jobs dynamically to available GPUs
- Generate video clips in parallel

2. Non-Functional Requirements

- Latency < 15 seconds per clip
- GPU utilization > 80% (pipeline and warp scheduling)
- Scalable to N GPUs (distributed system)

These requirements are defined specifically for the Core Engine pipeline and reflect the performance constraints observed during initial GPU experiments.

Functional Overview

The pipeline is designed around segment-level parallelism, where each script segment is treated as an independent unit of work.

During initial testing on a single GPU (RTX 4090), generating one video clip took approximately 40–50 seconds, which does not meet the system latency requirement. This led to the decision to process multiple segments concurrently across multiple GPUs.

Pipeline Flow:

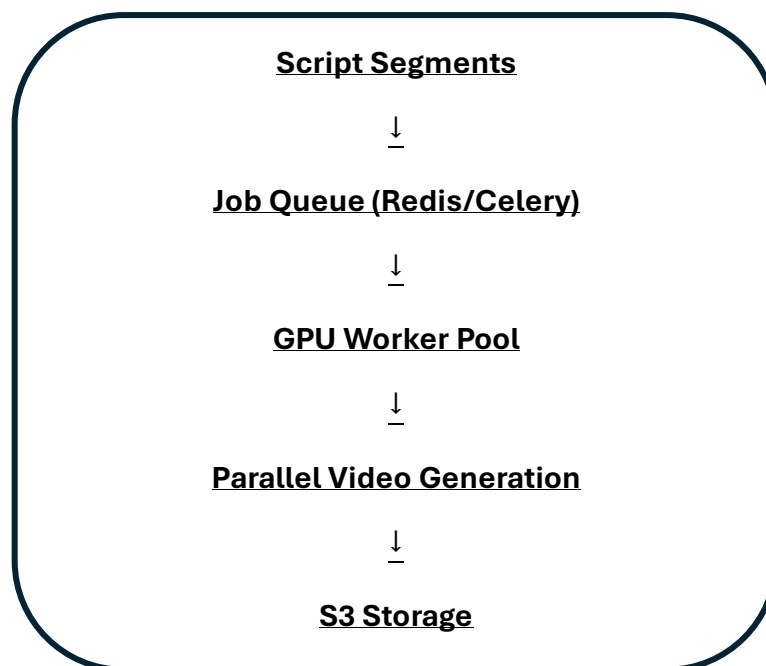


Figure 1: Parallel GPU Inference Pipeline (segment-level parallelism)

Each segment is dispatched as an independent task, allowing horizontal scaling across available GPU resources.

Configuration / External Interfaces

1. Dependencies

- PyTorch + CUDA
- Redis / Celery
- AnimateDiff / SVD
- AWS S3

2. External Interfaces

- REST API (job submission)
- Queue system (task dispatch)
- Storage API (video upload)

At the current stage, Redis + Celery is the primary queue system under testing, while Kafka is being considered as a future alternative for better scalability.

Debug

1. Logging

- Job Lifecycle logs (queued → running → completed)
- GPU assignment logs
- Error logs per segment

2. Counters

- GPU utilization %
- Jobs processed / sec
- Average latency
- System throughput / sec
- P95 / P99 latency (planned metrics, not yet fully implemented)

Implementation

This component is my primary responsibility in the project, focusing on HPC setup, GPU pipeline design, and performance optimization.

1. Design Proposal

The core idea of this design is segment-level parallelism. Each video segment is treated as an independent task and dispatched to available GPU workers. This design was chosen based on the following observations:

- a. Sequential inference on a single GPU is too slow (~45 seconds per clip)
- b. Frame-level parallelism would introduce high synchronization overhead
- c. Request-level parallelism does not fully utilize GPU resources

Therefore, segment-level parallelism provides a good balance between implementation complexity and performance gain.

2. Components

The system is composed as three main components

- a. Job dispatcher, which is responsible for receiving script segments from the upstream pipeline and pushing them into the task queue
- b. GPU worker, which is responsible for pulling task from queue and executing
 - Text-to-speech (TTS)
 - Video diffusion
 - Lip synchronization.
- c. Scheduler, which is responsible for assigning tasks based on GPU availability and memory usage

At the current stage, scheduling is simplified (first-available GPU), but future work will load-aware scheduling.

3. Current Implementation Status

At the time of writing, the full multi-GPU pipeline is not yet implemented. The following progress has been made:

- Local GPU environment setup (RTX 4090)
- AnimateDiff inference testing completed
- Baseline latency measured (~45 seconds per clip)
- FastAPI + async task system tested

The multi-GPU orchestration and scheduler are currently under development.

4. Future Subtasks

- GPU worker implementation
- Task queue integration (Kafka or RabbitMQ)
- Memory optimization

- Multi-GPU orchestration
- Benchmarking tools

Testing

1. General Approach

Testing will be conducted in two stages:

Stage 1: Single GPU baseline testing

- Measure latency per video clip
- Identify bottlenecks in diffusion and TTS

Stage 2: Multi-GPU testing (planned)

- Compare performance with increasing number of GPUs
- Measure throughput (video per minute)
- Evaluate GPU utilization

Both synthetic inputs and real academic PDF will be used.

2. Unit Tests

The following unit tests are planned or partially implemented:

- Task queue correctness (enqueue/dequeue)
- GPU worker execution validation
- Failure recovery (task retry)
- Output validation (video generation success)

At the current stage, unit testing is limited due to incomplete integration of the full pipeline.

Appendix

The following improvements are planned as the project progresses:

1. Dynamic load balancing
2. Auto-scaling GPU nodes
3. Kubernetes deployment